Table des matières

Cours 08 — Données semi-structurées 1
Objectifs 1
Théorie 2
Points clés à retenir 12
Exercices 13

Cours 08 — Données semi-structurées

Module 8/12 — Formation SQL avancé

Comment lire ce module — C'est le SQL de la donnée dont le schéma n'est pas décidé à l'avance : une réponse d'API, un événement, un webhook. On apprend à la fabriquer, à la ranger, à l'extraire, à l'aplatir, et surtout à savoir quand refuser de s'en servir. La théorie est ici ; les deux laboratoires (`Labo_08_OLAP_SQL.md` et `Labo_08_OLTP_SQL.md`) vous font faire l'aller et le retour : on construit un document à partir des tables, puis on le ré-exploite comme s'il venait du dehors.

Objectifs

À la fin de ce module, vous saurez :

- expliquer ce que « semi-structuré » veut dire, et d'où vient ce genre de donnée ;
- choisir entre `json` et `jsonb` sur PostgreSQL, et dire ce que le type `JSON` de l'entrepôt fait du document ;
- fabriquer un document à partir de lignes relationnelles, sur les deux moteurs ;
- extraire une valeur en sachant si vous récupérez du texte ou du document, ce qui est le piège numéro un ;
- typer une extraction sans la casser, et reconnaître un tri alphabétique déguisé en tri numérique ;
- vous servir de la famille normalisée `JSON_VALUE`, `JSON_QUERY`, `JSON_EXISTS` et `JSON_TABLE`, et dire ce qu'elle apporte que les flèches n'ont pas ;
- aplatir un tableau en lignes avec `LATERAL`, et retrouver le monde relationnel ;
- charger un fichier JSON en brut avant de le typer, et dire pourquoi c'est le bon ordre ;
- énoncer trois situations où stocker du JSON est une mauvaise décision.

© 2026 iAlexMG. Tous droits réservés.

Théorie

1. D'où vient le JSON dans une base

Le SQL suppose qu'on connaît les colonnes avant d'écrire la première ligne. C'est une force : le schéma est un contrat, et le moteur le fait respecter. Mais une partie de la donnée n'arrive pas comme ça.

La source	Ce qu'elle envoie	Pourquoi le schéma bouge
Une API tierce	une réponse par appel	le fournisseur ajoute un champ quand il veut, sans vous prévenir
Un flux d'événements	un message par action	chaque type d'événement porte des attributs différents
Un webhook	une notification par fait	le format suit les versions du produit qui l'émet
Un formulaire à champs variables	une réponse par soumission	les questions changent d'une campagne à l'autre

Le point commun : vous ne pouvez pas figer les colonnes, parce que vous ne décidez pas de la forme. « Semi-structuré » veut dire exactement cela. Il y a bien une structure, avec des clés, des valeurs et de l'imbrication, mais elle est portée par la donnée elle-même plutôt que par le catalogue de la base.

Le choix qui se présente alors est un vrai choix d'architecture, et il n'a que deux issues honnêtes. Soit on écrit un programme qui aplatit tout en colonnes avant l'insertion, et on casse le jour où un champ apparaît. Soit on stocke le document tel qu'il est arrivé, et on l'interroge en SQL. Ce module traite la seconde voie, et la section 9 dit à quelles conditions elle reste raisonnable.

2. Les types

Labo : `Labo_08_OLTP_SQL.md`, sous-partie 1.4.

Sur PostgreSQL, deux types coexistent, et le choix n'est pas anodin.

	`json`	`jsonb`
Stockage	le texte exact, tel qu'écrit	une forme binaire décomposée
Écriture	rapide, aucune analyse	plus lente, il faut analyser
Lecture	il faut ré-analyser à chaque accès	direct
Ordre des clés	conservé	perdu
Clés en double	conservées, la dernière gagne à la lecture	dédoublonnées à l'écriture

© 2026 iAlexMG. Tous droits réservés.

	json	jsonb
Espaces et mise en forme	conservés	perdus
Indexable	non	oui, avec GIN

En pratique, on écrit `jsonb`. Le seul cas qui justifie `json` est celui où la fidélité au texte d'origine compte pour des raisons légales ou d'audit, parce qu'il faut pouvoir prouver ce qui a été reçu, octet pour octet.

Sur l'entrepôt, il n'y a qu'un type, `JSON`. Il accueille indifféremment un objet, un tableau ou une valeur simple, et c'est ce que rendent toutes les fonctions du module : `JSON_OBJECT('a', 1)` sort en `JSON`, et `'{"a":1}'::JSON` aussi. Il n'y a donc pas de choix à faire, contrairement au transactionnel.

Ce type garde le texte tel qu'il est arrivé : un document écrit avec des espaces superflus et une clé en double ressort à l'identique, mesuré, ce qui le rapproche du `json` de PostgreSQL plutôt que de son `jsonb`. Chaque lecture d'un chemin repart donc du texte, et elle se paie : sur 1,2 million de documents, extraire un montant coûte une trentaine de fois la lecture de la même valeur rangée dans une colonne ordinaire ou dans un `STRUCT`, mesuré.

Portabilité — Les grands entrepôts font autrement. Snowflake, d'après sa documentation, extrait à l'écriture les éléments d'un `VARIANT` en colonnes, jusqu'à 200 par partition, si bien qu'interroger un élément extrait ne lit que sa colonne. Un élément qui porte ne serait-ce qu'un `null`, ou des types différents d'une ligne à l'autre, n'est pas extrait, et sa lecture repart du document entier.

Retenez l'idée générale : l'entrepôt de la formation ne fait rien pour un chemin qu'on relit, et les grands entrepôts n'aident que les chemins réguliers. Aucun ne dispense de réfléchir à la forme des documents, et la section 8 y revient.

3. Fabriquer du JSON

Labo : `Labo_08_OLTP_SQL.md` et `Labo_08_OLAP_SQL.md`, sous-partie 1.1.

Avant d'interroger un document, il est utile d'en construire un. Cela oblige à comprendre sa forme, et cela vous donnera la matière des laboratoires sans rien télécharger.

Sur PostgreSQL :

```
SELECT jsonb_build_object(
         'commande', h.salesorderid,
         'date',     h.orderdate,
         'client',   h.customerid,
         'lignes',   (SELECT jsonb_agg(jsonb_build_object('produit', d.productid,
                                                          'qte',     d.orderqty,
                                                          'prix',    d.unitprice))
                      FROM   sales.salesorderdetail AS d
                      WHERE  d.salesorderid = h.salesorderid)
       ) AS document
```


© 2026 iAlexMG. Tous droits réservés.

```
  FROM   sales.salesorderheader AS h
  WHERE  h.salesorderid = 43659;
```

`jsonb_build_object` prend des arguments par paires, clé puis valeur. `jsonb_agg` rassemble en tableau les lignes d'un groupe. Notez que la sous-requête est corrélée : c'est le geste du Cours 07, réemployé ici pour imbriquer les lignes dans leur commande.

Sur l'entrepôt, les mêmes idées avec d'autres noms :

```
  SELECT JSON_OBJECT(
           'commande', f.SalesOrderNumber,
           'client',   f.CustomerKey,
           'lignes',   JSON_GROUP_ARRAY(JSON_OBJECT('produit', f.ProductKey,
                                                    'qte',     f.OrderQuantity,
                                                    'montant', f.SalesAmount))
         ) AS document
  FROM   FactInternetSales AS f
  GROUP  BY f.SalesOrderNumber, f.CustomerKey;
```

La correspondance est terme à terme : `jsonb_build_object` devient `JSON_OBJECT`, et `jsonb_agg` devient `JSON_GROUP_ARRAY`. Seuls les noms changent, et c'est la bonne nouvelle de ce module : la manière de penser un document ne dépend pas du moteur.

Piège — Le sort d'une clé dont la valeur est nulle, lui, dépend du moteur, et c'est le premier écart à connaître. Nos deux terrains sont d'accord : `JSON_OBJECT('a', 1, 'b', NULL)` rend `{"a":1,"b":null}`, avec les deux clés, comme `jsonb_build_object` côté PostgreSQL. D'autres moteurs font l'inverse et écartent la clé sans le dire, dont Snowflake d'après sa documentation, où il faut écrire `OBJECT_CONSTRUCT_KEEP_NULL` pour la garder. La conséquence est la même partout : deux documents fabriqués à partir de lignes également incomplètes n'ont pas forcément les mêmes clés, et un test de présence de clé ne dit pas ce qu'on croit. La section 4 montre que ce `null` n'est d'ailleurs pas non plus le `NULL` de SQL.

4. Extraire

Labo : `Labo_08_OLAP_SQL.md`, sous-partie 3.1 ; `Labo_08_OLTP_SQL.md`, sous-partie 2.1.

C'est ici que se trouve la faute la plus commune du module, et elle mérite qu'on s'y arrête longuement.

Sur PostgreSQL, quatre opérateurs, qui vont deux par deux.

Opérateur	Ce qu'il prend	Ce qu'il rend
`->`	une clé ou un indice	du `jsonb`
`->>`	une clé ou un indice	du **texte**
`#>`	un chemin, sous forme de tableau	du `jsonb`

© 2026 iAlexMG. Tous droits réservés.

Opérateur	Ce qu'il prend	Ce qu'il rend
`#>>`	un chemin, sous forme de tableau	du **texte**

```sql
SELECT document -> 'client'                    AS en_jsonb,    -- "Bilodeau"
       document ->> 'client'                   AS en_texte,    -- Bilodeau
       document #> '{lignes,0,produit}'        AS en_jsonb,
       document #>> '{lignes,0,produit}'       AS en_texte
FROM   commande_json;
```

La flèche simple garde le document, donc une chaîne de caractères revient **avec ses guillemets**. La flèche double rend du texte nu. La règle de choix est mécanique : on emploie la flèche simple tant qu'on continue de descendre dans le document, et la flèche double au dernier cran, quand on veut enfin la valeur.

PostgreSQL propose aussi `jsonb_path_query`, qui applique une expression de chemin proche de celles qu'on écrit en JavaScript. C'est plus puissant pour filtrer à l'intérieur d'un tableau, et plus verbeux pour un simple accès.

Ces opérateurs sont l'héritage de PostgreSQL, antérieurs à la norme. Depuis les versions 16 et 17, il en existe une seconde famille, celle de la norme SQL/JSON, qui porte les mêmes noms sur tous les moteurs qui l'ont adoptée, et qui dit le chemin, le type et le comportement en cas d'échec dans une seule expression.

Fonction	Ce qu'elle rend	Ce qu'elle remplace
`JSON_VALUE(doc, '$.chemin' RETURNING type)`	une valeur simple, déjà typée	`(doc #>> '{chemin}')::type`
`JSON_QUERY(doc, '$.chemin')`	un morceau de document, objet ou tableau	`doc #> '{chemin}'`
`JSON_EXISTS(doc, '$.chemin')`	vrai ou faux, avec filtre possible	`doc ? 'clé'`, en plus expressif
`doc IS JSON [OBJECT|ARRAY|SCALAR]`	vrai ou faux, la forme du document	rien, il fallait tenter la conversion

```sql
SELECT JSON_VALUE(document, '$.client' RETURNING int)   AS client,
       JSON_QUERY(document, '$.lignes[0]')              AS premiere_ligne,
       JSON_EXISTS(document, '$.lignes[*] ? (@.qte > 3) ') AS grosse_ligne
FROM   commande_json;
```

Trois choses valent le détour, toutes mesurées sur le PostgreSQL 18 du poste. Le `RETURNING` évite la conversion séparée, donc l'oubli de conversion de la section 5. Le filtre `? (@.qte > 3)` de `JSON_EXISTS` interroge l'intérieur d'un tableau sans l'aplatir. Et l'échec ne fait plus tomber la requête, ce qui est le sujet de la section suivante.

Portabilité — L'intérêt de cette famille est justement qu'elle est normalisée, là où les flèches `->` et `->>` sont une invention de PostgreSQL que DuckDB a reprise par commodité. D'après leur documentation, Oracle et MySQL portent `JSON_VALUE` et `JSON_TABLE` sous ces noms, et SQL Server `JSON_VALUE` et `JSON_QUERY`. DuckDB n'en a que la moitié, et sous une autre forme, mesuré : il connaît `JSON_VALUE` et `JSON_EXISTS` comme des fonctions ordinaires à deux arguments, sans

© 2026 iAlexMG. Tous droits réservés.

clause `RETURNING`, si bien que `JSON_VALUE('{"montant": "12.5"}', '$.montant')` rend `"12.5"` avec ses guillemets et laisse la conversion à votre charge ; il n'a ni `JSON_QUERY` ni `JSON_TABLE`, et la syntaxe normalisée de la clause `RETURNING` y rend une erreur. La voie analytique de ce cours s'en tient donc aux flèches et aux fonctions `JSON_*` du moteur. Retenez la règle de choix plutôt que la liste : pour du code appelé à changer de moteur, la famille normalisée est le meilleur pari ; pour du code qui reste ici, les flèches sont plus courtes et, on va le mesurer à la section 6, plus rapides.

Sur l'entrepôt, la notation est la même, et c'est encore une bonne nouvelle : les deux flèches de PostgreSQL y fonctionnent à l'identique, avec la même règle de choix.

```sql
SELECT charge -> 'commande'                AS en_json,   -- "SO43697", guillemets compris
       charge ->> 'commande'               AS en_texte,  -- SO43697
       charge -> 'lignes' -> 0 -> 'montant' AS en_json,
       charge -> '$.lignes[0].montant'     AS pareil,    -- le chemin en une flèche
       JSON_EXTRACT_STRING(charge, '$.lignes[0].montant') AS en_texte_aussi
FROM   commande_json;
```

Deux ajouts par rapport au transactionnel. La flèche accepte un chemin complet écrit d'un trait, `'$.lignes[0].montant'`, ce qui évite d'enchaîner les crans ; et `-> 0` indexe un tableau par sa position, comptée à partir de zéro. Les fonctions `JSON_EXTRACT` et `JSON_EXTRACT_STRING` font la même chose que les deux flèches, sous une forme nommée.

Piège — Une valeur extraite par la flèche simple reste du `JSON`, donc une chaîne s'affiche **avec ses guillemets**. Une comparaison comme `WHERE charge -> 'commande' LIKE 'SO437%'` rend donc zéro ligne, parce qu'elle compare `"SO43701"`, guillemets compris, à un motif qui commence par `S`. Il faut la flèche double. Ce défaut ne produit aucune erreur : la requête renvoie zéro ligne, exactement comme le `NOT IN` du Cours 07. Vous commencez à reconnaître la famille.

Deux détails d'écriture, mesurés sur l'entrepôt, qui vous éviteront de chercher. La flèche se lie moins fort que l'égalité : sans parenthèses, `charge -> 'commande' = 'SO43701'` est lu `charge -> ('commande' = 'SO43701')` et le moteur répond `No function matches … json_extract(JSON, BOOLEAN)`. Et comparer une valeur `JSON` à un texte échoue aussi, sur `Malformed JSON at byte 0 of input`, parce que le moteur essaie de convertir le texte en document. Ces deux-là, au moins, sont bruyants.

5. Typer

Labo : `Labo_08_OLAP_SQL.md`, sous-parties 3.2 et 3.3 ; `Labo_08_OLTP_SQL.md`, sous-parties 2.2 et 2.3.

Une extraction ne rend jamais un nombre. Elle rend du texte avec la flèche double, un morceau de document avec la flèche simple, et c'est vrai des deux côtés. C'est à vous de convertir, et l'oubli se paie de deux façons.

```sql
-- PostgreSQL : sans conversion, le tri est ALPHABÉTIQUE.
SELECT document ->> 'montant' FROM commande_json ORDER BY document ->> 'montant' DESC;
-- Renvoie 99 avant 1000, parce que « 9 » vient après « 1 ».

-- Avec conversion, le tri est numérique.
SELECT (document ->> 'montant')::numeric FROM commande_json
ORDER  BY (document ->> 'montant')::numeric DESC;
```


© 2026 iAlexMG. Tous droits réservés.

Le tri alphabétique déguisé en tri numérique est le mode de défaillance le plus fréquent, et le plus silencieux : le résultat est trié, il a l'air correct, et il est faux. La même chose vaut pour une comparaison, où `'99' > '1000'` est vrai en texte.

La seconde façon de se faire piéger est la conversion qui échoue. Un document sur dix mille contient `"montant": "n/a"`, et toute la requête tombe. D'où les fonctions tolérantes :

Moteur	Conversion stricte	Conversion tolérante
PostgreSQL	`(x)::numeric`, échoue sur une valeur invalide	`JSON_VALUE(doc, '$.x' RETURNING numeric)`, qui rend `NULL`
L'entrepôt	`x::DECIMAL(19,4)`, échoue sur `Could not convert string "n/a"`	`TRY_CAST(x AS DECIMAL(19,4))`, qui rend `NULL`

Les deux terrains ont donc leur conversion tolérante, et c'est ce qui compte : une seule valeur fautive dans un lot de cent mille documents fait tomber une requête stricte, et il faut ensuite aller chercher laquelle. Avec la tolérante, elle devient un `NULL` qu'on compte, qu'on isole et qu'on traite à part.

Les deux écritures ne sont pourtant pas jumelles, et l'écart est à l'avantage de la norme. `TRY_CAST` ne sait dire qu'une chose, la conversion a échoué. `JSON_VALUE` distingue deux échecs différents, et les traite séparément : la valeur était là mais inconvertible (`ON ERROR`), ou le chemin ne menait à rien (`ON EMPTY`). Mesuré sur trois documents, `{"montant": "123.45"}`, `{"montant": "n/a"}` et `{"autre": 1}` :

```
SELECT JSON_VALUE(d, '$.montant' RETURNING numeric)                  AS tolerant,
       JSON_VALUE(d, '$.montant' RETURNING numeric DEFAULT -1 ON ERROR) AS si_invalide,
       JSON_VALUE(d, '$.montant' RETURNING numeric DEFAULT -9 ON EMPTY) AS si_absent
FROM   essai;
```

La première colonne rend 123,45 puis deux `NULL`, sans distinguer. La deuxième ne remplace que la valeur inconvertible, la troisième ne remplace que la clé absente. Sans clause, l'échec est silencieux dans les deux cas ; avec `ERROR ON ERROR`, la requête tombe, ce qui reste le bon choix quand une valeur invalide doit arrêter un chargement plutôt que de le traverser. Le défaut est donc la tolérance, et c'est un choix qu'on fait, pas un hasard.

Piège — Une clé présente dont la valeur est le `null` de JSON n'est ni une erreur ni un vide : `JSON_VALUE` rend `NULL`, `JSON_EXISTS` rend vrai, et la flèche `->>` rend `NULL` elle aussi. Une comparaison qui compte les valeurs écartées par `WHERE doc ->> 'montant' IS NOT NULL` passe donc à côté de ces documents, et les confond avec ceux où la clé manque. C'est le même piège qu'au début de ce cours, vu d'un autre angle.

Rappel — `CAST`, `::` et `TRY_CAST` sont présentés au Cours 02, section 4 ; on ne regarde ici que ce qu'ils deviennent sur un document.

Le réflexe à prendre : sur une donnée dont vous ne maîtrisez pas la source, la conversion tolérante est le défaut, et l'on compte ensuite combien de valeurs sont tombées à `NULL`. Sur un flux de documents, l'enjeu grossit : une conversion stricte qui échoue sur un lot entier à cause d'une ligne est une panne.

6. Aplatir

Labo : `Labo_08_OLAP_SQL.md`, sous-partie 3.4 ; `Labo_08_OLTP_SQL.md`, sous-partie 2.4.

© 2026 iAlexMG. Tous droits réservés.

Un tableau imbriqué dans un document contient plusieurs valeurs. Pour revenir au monde relationnel, il faut en faire des lignes. C'est exactement le geste du Cours 07 : une sous-requête qui produit plusieurs lignes pour chaque ligne de la table écrite à sa gauche (ici, chaque document de `commande_json`), donc `LATERAL`.

Sur PostgreSQL :

```
SELECT c.document ->> 'commande'      AS commande,
       ligne ->> 'produit'            AS produit,
       (ligne ->> 'qte')::int         AS qte
FROM   commande_json AS c,
       LATERAL jsonb_array_elements(c.document -> 'lignes') AS ligne;
```

`jsonb_array_elements` est une fonction qui renvoie un ensemble de lignes, une par élément du tableau. Le `LATERAL` lui donne accès à `c.document`. Il existe aussi `jsonb_array_elements_text`, qui rend directement du texte quand le tableau ne contient que des valeurs simples.

La norme SQL/JSON a mieux, et c'est la pièce qui manquait au tableau de la section 4 : `JSON_TABLE` déclare d'un coup le chemin du tableau, les colonnes voulues, leur type et leur chemin à l'intérieur de chaque élément. Il rend une table, qu'on emploie comme n'importe quelle table.

```
SELECT c.document ->> 'commande' AS commande, l.*
FROM   commande_json AS c,
       JSON_TABLE(c.document, '$.lignes[*]'
                  COLUMNS (position FOR ORDINALITY,
                           produit int     PATH '$.produit',
                           qte     int     PATH '$.qte',
                           prix    numeric PATH '$.prix')) AS l;
```

Comparez avec le bloc précédent, qui fait la même chose. Trois corvées disparaissent, toutes mesurées au labo. Les conversions `::int` et `::numeric` sont remontées dans la déclaration, donc on ne peut plus en oublier une. `FOR ORDINALITY` donne la position dans le tableau, que `jsonb_array_elements` fait payer d'un `WITH ORDINALITY`. Et une valeur inconvertible rend `NULL` dans sa colonne sans faire tomber la requête, comme `JSON_VALUE` de la section 5, avec la même clause `DEFAULT … ON ERROR` si l'on veut marquer le coup.

Ce n'est pas gratuit. Sur les 31 465 commandes du labo, le même calcul de total écrit avec `JSON_TABLE` prend environ une fois et demie le temps de la version aux flèches, cinq exécutions de chaque. La norme achète la clarté, le typage et le voyage entre moteurs ; elle ne fait pas gagner de temps.

Sur l'entrepôt, la fonction s'appelle `JSON_EACH` :

```
SELECT c.charge ->> 'commande'                  AS commande,
       ligne.key::INTEGER                        AS position,
       (ligne.value ->> 'produit')::INTEGER      AS produit,
       (ligne.value ->> 'qte')::INTEGER          AS qte
FROM   commande_json AS c,
       JSON_EACH(c.charge -> 'lignes') AS ligne;
```


© 2026 iAlexMG. Tous droits réservés.

Le mot `LATERAL` ne figure pas ici, et c'est le rappel du Cours 07, section 8 : l'entrepôt le rend facultatif, alors que PostgreSQL l'exige. L'écrire quand même ne coûte rien et rend la requête lisible des deux côtés.

`JSON_EACH` ne rend pas seulement la valeur : elle rend aussi une colonne `key`, qui porte la position dans le tableau, comptée à partir de zéro, ce qui permet de conserver l'ordre d'origine. Sur un objet plutôt qu'un tableau, cette même colonne porte le nom de la clé, comme le montre `JSON_EACH('{"x":1,"y":2}')`, qui rend deux lignes, `x` et `y`.

Portabilité — `JSON_TABLE` est la forme normalisée de ce geste, et son pendant chez Snowflake est `LATERAL FLATTEN`, qui rend d'après sa documentation la clé, le chemin et l'indice, et accepte deux options : `OUTER => TRUE`, qui conserve le document même quand son tableau est vide, et `RECURSIVE => TRUE`, qui descend dans toute l'imbrication d'un coup. La première a un équivalent exact ici, mesuré : un tableau vide fait bien disparaître son document sur nos deux terrains, et une jointure externe le ramène, `LEFT JOIN LATERAL JSON_TABLE(…) ON true` sur PostgreSQL. C'est le `LEFT` du Cours 07, une troisième fois. La seconde n'a pas d'équivalent : une imbrication profonde se descend ici niveau par niveau.

Une fois aplati, vous êtes revenu au relationnel et tout le reste de la formation s'applique.

7. Charger

Labo : `Labo_08_OLAP_SQL.md`, partie 2.

Sur un entrepôt, un fichier JSON ne se charge pas colonne par colonne. On le charge **en brut**, dans une seule colonne `JSON`, et on type ensuite dans une vue ou une table dérivée.

```sql
-- Une table d'accueil a une seule colonne, remplie par la lecture du fichier.
CREATE OR REPLACE TABLE commande_brute AS
SELECT json AS charge FROM READ_JSON_OBJECTS('commandes.json');
```

Rappel — `CREATE OR REPLACE TABLE`, et ce que le type d'une colonne fait à la donnée, sont présentés au Cours 03, section 2.

`READ_JSON_OBJECTS` rend chaque objet du fichier tel quel, sans rien deviner. Sa voisine `READ_JSON`, sans le suffixe, fait autre chose : elle infère un schéma et rend des colonnes typées, ce qui est commode et risqué, un montant en `DECIMAL(19,4)` pouvant y ressortir en `DOUBLE`, un type approché. Pour accueillir un document tel quel, prenez `READ_JSON_OBJECTS` ; pour obtenir des colonnes, déclarez leur type plutôt que de le laisser deviner.

Une option décide de tout : celle qui dit la forme du fichier. Un fichier qui contient `[{...}, {...}, {...}]` est un seul tableau. Lu avec `format = 'array'`, il vous donne une ligne par élément, ce qui est presque toujours ce que vous voulez ; lu avec `format = 'unstructured'`, il vous donne **une ligne** contenant tout le fichier. Sans l'option, DuckDB devine la forme, et devine juste sur un fichier propre, mesuré : le danger n'y est pas l'oubli, c'est la forme imposée à tort, celle d'un pipeline réglé pour une autre source. Le pire des trois cas est le silence : la lecture non structurée rend sa ligne unique sans un mot, alors que la lecture ligne par ligne, elle, refuse le fichier en le nommant. Sur Snowflake, la même option s'appelle `STRIP_OUTER_ARRAY` et reste désactivée tant qu'on ne la demande pas, d'après sa documentation : là-bas, c'est l'oubli qui charge le tableau entier dans une seule ligne, l'oubli classique d'un premier chargement JSON.

© 2026 iAlexMG. Tous droits réservés.

Pourquoi charger en brut d'abord. C'est le même raisonnement que la couche `RAW` d'un entrepôt, et il tient en une phrase : le jour où le fournisseur ajoute un champ, la donnée brute l'a déjà, la colonne typée non. Si vous typez au chargement, vous perdez définitivement ce que vous n'aviez pas prévu, et il faut redemander l'historique à la source. Si vous chargez brut, ajouter le champ revient à modifier une vue, et l'historique complet est là.

Le carnet d'ordres et le webhook — Un message d'exchange illustre tout ce paragraphe. Il porte un horodatage, un symbole, un côté, un prix, une quantité, et un nombre variable de champs annexes selon la version de l'API et le type d'événement. Le stocker en brut coûte quelques octets de plus et vous sauve le jour où l'exchange ajoute un champ de frais, une profondeur de carnet supplémentaire ou un identifiant de session. Typer au chargement fait gagner une jointure et vous fait perdre l'historique du champ que vous n'aviez pas anticipé.

8. Performer

Labo : `Labo_08_OLTP_SQL.md`, partie 3.

Sur PostgreSQL, le semi-structuré est lent par défaut, parce qu'aucun index ne couvre l'intérieur d'un document. La parade est l'index GIN, pour *generalized inverted index*, l'index inversé généralisé : il indexe chaque clé et chaque valeur du document plutôt que le document entier.

```
CREATE INDEX idx_doc ON commande_json USING GIN (document);
```

Il sert trois opérateurs qui n'ont pas d'équivalent hors du JSON :

Opérateur	Se lit	Exemple
`@>`	contient	`document @> '{"pays": "Canada"}'`
`?`	possède la clé	`document ? 'remise'`
`?&` et `?|`	possède toutes ces clés, ou l'une d'elles	`document ?& array['prix','qte']`

Deux avertissements pratiques. L'index GIN est gros et coûteux à mettre à jour, ce qui en fait un mauvais choix sur une table très écrite. Et il **n'accélère pas** un accès par flèche du type `document ->> 'pays' = 'Canada'` : pour ce cas, il faut soit reformuler avec `@>`, soit créer un index sur l'expression elle-même.

Une variante mérite d'être connue. La classe d'opérateurs `jsonb_path_ops`, qui s'écrit `USING GIN (document jsonb_path_ops)`, n'indexe que les chemins et leurs valeurs : elle sert `@>`, pas `?`. Mesuré sur les 31 465 documents du labo, elle pèse 3 736 ko contre 4 088 ko pour l'index ordinaire, sert `@>` par la même descente, et laisse `document ? 'remise'` au parcours séquentiel. Quand les requêtes ne cherchent que par contenance, c'est elle qu'on crée.

Sur l'entrepôt, il n'y a pas d'index qui couvre l'intérieur d'un document, et il n'en faut pas au même titre : le Cours 04 a montré qu'un index n'y change ni le plan ni le temps, même sur des colonnes ordinaires. Ce qui coûte, ici, c'est de redescendre dans le document à chaque lecture.

La parade ne change donc pas d'un moteur à l'autre, et c'est la conclusion à retenir de ce module : quand une valeur extraite est lue en permanence, on cesse de l'extraire à chaque fois et on la matérialise, dans une table dérivée ou une vue calculée une fois. Les grands entrepôts en font une partie automatiquement, Snowflake extrayant les éléments en colonnes d'après sa documentation, mais cette aide a les limites que la même documentation écrit : pas plus de 200

© 2026 iAlexMG. Tous droits réservés.

éléments extraits par partition, et aucun élément qui porte un `null` ou des types mêlés. Autrement dit, elle vous dispense du travail facile et vous laisse le difficile.

9. Quand ne pas utiliser le JSON

Labo : `Labo_08_OLAP_SQL.md`, sous-partie 3.5 ; `Labo_08_OLTP_SQL.md`, partie 4.

C'est la section la plus utile du module, parce que le semi-structuré est un outil dont l'abus ne se voit pas tout de suite.

La colonne fourre-tout. Une table a huit colonnes propres et une neuvième nommée `donnees`, où l'on met « le reste ». Six mois plus tard, la moitié de l'information métier y est. Le signe qui ne trompe pas : personne ne sait dire ce qu'elle contient sans lire du code.

Le schéma qu'on refuse de décider. Le JSON permet de reporter la conversation difficile avec le métier sur ce qu'est vraiment une commande. Reporter n'est pas résoudre, et le coût revient multiplié : chaque lecteur suivant devra deviner, et deux lecteurs devineront différemment.

Le champ stable rangé en JSON. Un identifiant, une date, un montant qui figurent dans tous les documents depuis deux ans ne sont plus du semi-structuré, ce sont des colonnes. Les laisser dans le document coûte une extraction et une conversion à chaque lecture, pour rien.

La règle qui découle des trois : **le JSON est bon pour ce qui varie, mauvais pour ce qui est stable**. Le motif sain consiste à garder le document brut pour l'audit et la reprise, et à matérialiser en colonnes ce qui s'est stabilisé. C'est ce que fait un entrepôt bien tenu entre sa couche brute et sa couche exploitée.

Une dernière raison de s'en méfier, que la section 4 a déjà montrée : les modes de défaillance du semi-structuré sont silencieux. Un chemin qui n'existe pas renvoie `NULL`, une valeur extraite sans la bonne flèche renvoie zéro ligne, une extraction non typée trie alphabétiquement. Aucune de ces trois fautes ne produit d'erreur.

10. Sur les deux terrains

Labo : `Labo_08_OLAP_SQL.md`, partie 4 ; `Labo_08_OLTP_SQL.md`, partie 5.

Sujet	PostgreSQL (voie OLTP)	DuckDB (voie OLAP)
Le type	`json` et `jsonb`, on écrit `jsonb`	`JSON`, un seul, pas de choix à faire
Fabriquer	`jsonb_build_object`, `jsonb_agg`	`JSON_OBJECT`, `JSON_GROUP_ARRAY`
Clé de valeur nulle	conservée	conservée aussi
Extraire	`->` rend du `jsonb`, `->>` du texte ; la norme ajoute `JSON_VALUE` et `JSON_QUERY`	les mêmes flèches, plus le chemin d'un trait `-> '$.a.b'` ; `JSON_VALUE` sans `RETURNING`
Typer	`::numeric`, qui échoue, ou `JSON_VALUE … RETURNING`, qui rend `NULL`	`::DECIMAL`, ou `TRY_CAST`, qui rend `NULL`

© 2026 iAlexMG. Tous droits réservés.

Sujet	PostgreSQL (voie OLTP)	DuckDB (voie OLAP)
Aplatir	`jsonb_array_elements` avec `LATERAL`, ou `JSON_TABLE`, qui type et numérote	`JSON_EACH`, qui rend aussi la position, et sans `LATERAL` obligatoire
Tableau vide	rien, sauf `LEFT JOIN LATERAL`	rien non plus, même parade
Indexer	GIN, avec `@>`, `?`, `?&`	pas d'index utile : on matérialise ce qu'on relit
Charger un fichier	`COPY` de PostgreSQL, ou insertion applicative	`READ_JSON_OBJECTS`, avec l'option `format` qui dit la forme du fichier

Le tableau se lit dans les deux sens. Colonne par colonne, les noms changent presque tous. Ligne par ligne, les idées ne changent presque jamais : un type qui accueille le document, une fabrique, deux façons d'extraire dont l'une garde les guillemets, une conversion à ne pas oublier, un aplatissement, et la question de ce qu'on matérialise. Apprenez les lignes, cherchez les noms au besoin.

Le piège partagé, et il n'a rien d'un détail de moteur : sur les deux terrains, l'extraction ne rend jamais un nombre, et l'oubli de conversion produit un résultat faux plutôt qu'une erreur.

Points clés à retenir

- Semi-structuré veut dire que la structure est portée par la donnée, pas par le catalogue. On l'emploie quand on ne décide pas de la forme, jamais pour éviter de décider la sienne.
- Sur PostgreSQL on écrit `jsonb`. `json` ne se justifie que si la fidélité au texte reçu doit être prouvable.
- Sur l'entrepôt, un seul type, `JSON`, et les mêmes deux flèches que sur le transactionnel. Ce module ne demande donc presque aucun dialecte nouveau, seulement d'autres noms de fonctions.
- `->` garde le document, `->>` rend le texte. On descend avec la flèche simple et on sort avec la double, au dernier cran seulement.
- Une valeur gardée en document s'affiche avec ses guillemets. Comparer sans la flèche double renvoie zéro ligne, sans erreur.
- Une extraction ne rend jamais un nombre. Sans conversion, le tri est alphabétique et le résultat est faux en silence.
- Sur une source non maîtrisée, la conversion tolérante est le défaut, puis on compte les valeurs tombées à `NULL`. Les deux terrains l'ont : `TRY_CAST` sur l'entrepôt, `JSON_VALUE … RETURNING` sur PostgreSQL depuis la version 16, qui distingue en plus la valeur inconvertible de la clé absente.
- Aplatir un tableau, c'est le `LATERAL` du Cours 07. `jsonb_array_elements` ou `JSON_TABLE` d'un côté, `JSON_EACH` de l'autre, ces deux dernières rendant en plus la position dans le tableau. Des deux côtés, un tableau vide fait disparaître son document, sauf jointure externe.

© 2026 iAlexMG. Tous droits réservés.

- On charge en brut d'abord, on type ensuite. Le jour où le fournisseur ajoute un champ, la donnée brute l'a déjà, la colonne typée non.
- L'option de forme décide si un fichier `[{…},{…}]` devient une ligne ou plusieurs : `format = 'array'` sur l'entrepôt, `STRIP_OUTER_ARRAY` sur Snowflake. DuckDB devine la forme qu'on ne lui dit pas ; imposer la mauvaise, `unstructured`, charge une ligne par fichier sans rien dire.
- L'index GIN sert `@>`, `?` et `?&`, mais pas un accès par flèche. Il est gros et coûteux à maintenir.
- Le JSON est bon pour ce qui varie, mauvais pour ce qui est stable. Un champ présent dans tous les documents depuis deux ans est une colonne.
- Les trois fautes du module sont silencieuses : chemin inexistant, comparaison sans conversion, tri non typé. Aucune ne produit d'erreur.

Exercices

Les exercices de ce module et leur corrigé vivent dans `Exercices_corriges_08_SQL.md`. Faites-les après les laboratoires : plusieurs énoncés supposent que le document a été fabriqué.

© 2026 iAlexMG. Tous droits réservés.